

# Build RAG with LangGraph

---

*From raw PDF to a production grade answer engine, one node at a time.*

# What is RAG

## BASICS

Retrieval Augmented Generation. Instead of hoping the model already knows your data, you fetch the right pieces of your own documents first, then ask the model to answer using only that context.

### THE LOOP



### WHY IT MATTERS



No retraining needed



Always fresh data



Fewer hallucinations



Traceable citations

### WHERE TEAMS USE IT



Document QA bots



Customer support



Legal and policy search



Internal knowledge

# Why LangGraph

## TOOLING

LangChain gives you the components. LangGraph gives you the control flow. You model your RAG app as nodes and edges that read and write one shared state object.

### LANGCHAIN CHAIN

- Runs start to finish
- One direction only
- Hard to retry a step
- State hidden inside

### LANGGRAPH GRAPH

- Loops and branches
- Retry any single node
- Explicit shared state
- Memory and human review

## WHAT YOU NEED BEFORE YOU START

 Python 3.10 or newer

 An LLM API key

 A vector store

 Your source documents

Real RAG is never one straight line. You grade results, rewrite bad queries and try again. That is exactly what a graph is for.

# One Kit to Clear Your AI Interview

KIT

After seeing how AI interviews are evolving, I created an **AI Interview Kit**. It covers the areas companies actually test:



AI fundamentals



AI agents



Transformers in depth



Memory systems



LLM architectures



Databases



Prompt engineering



Backend for AI systems



Fine tuning



Deployment strategies



RAG systems



GenAI system design

Everything structured to help you prepare.

↓ [Check the caption to enroll](#)

# What We Are Building

SETUP

A self correcting PDF question answering graph. It ingests documents once, then serves answers with retrieval, reranking, grading and a rewrite loop.

## INGESTION, RUN ONCE

PDF Load

,

Chunk

,

Embed

,

Vector DB

## QUERY GRAPH, RUNS PER QUESTION

Retrieve

,

Rerank

,

Grade

,

Generate

## INSTALL THE STACK

```
# core graph and LLM layer
pip install langgraph langchain langchain-openai

# loaders, splitting, vector store
pip install pypdf langchain-community langchain-chroma

# reranking and evaluation
pip install sentence-transformers ragas datasets
```

# Define the Graph State

BACKBONE

## WHAT THIS STEP IS

State is the shared dictionary every node reads from and writes to. Define it first, because every node signature depends on it.

```
from typing import TypedDict, List
from langchain_core.documents import Document

class RAGState(TypedDict):
    question: str          # original user question
    query: str              # rewritten search query
    docs: List[Document]   # retrieved chunks
    answer: str             # final generated answer
    grade: str              # useful or not useful
    retries: int            # loop guard
```

## READING THE CODE



Each key is one piece of data a node produces



A node returns a partial dict and LangGraph merges it



retries stops the rewrite loop from running forever

# Load and Scrape the PDF

INGEST

## WHAT THIS STEP IS

Turn binary files into clean text objects. Every page becomes a Document with page content plus metadata you will need later for citations.

```
from langchain_community.document_loaders import (
    PyPDFLoader, PyPDFDirectoryLoader)

# one file
docs = PyPDFLoader("handbook.pdf").load()

# or a whole folder
docs = PyPDFDirectoryLoader("./data").load()

for d in docs:
    d.metadata["source"] = d.metadata.get("source", "unknown")

print(len(docs), docs[0].page_content[:200])
```

## WATCH OUT FOR



Scanned PDFs need OCR



Tables lose layout

Garbage text here means garbage answers later. Always print a sample before moving on.

# Chunk the Documents

SPLIT

## WHAT THIS STEP IS

A full page is too big to embed well. Split text into overlapping windows so each chunk carries one coherent idea and a little context from its neighbour.

```
from langchain_text_splitters import (  
    RecursiveCharacterTextSplitter)  
  
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=800,  
    chunk_overlap=120,  
    separators=["\n\n", "\n", ". ", " "],  
    add_start_index=True,  
)  
  
chunks = splitter.split_documents(docs)  
print(len(chunks))
```

## HOW TO PICK THE NUMBERS



400 to 800 characters suits most prose documents



Overlap of 10 to 20 percent keeps sentences from breaking



Code and tables need larger chunks and structure aware splitters

# Create Embeddings

## VECTORS

### WHAT THIS STEP IS

Convert each chunk into a list of numbers that captures meaning. Two texts about the same idea land close together, which is what makes semantic search possible.

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small")

vec = embeddings.embed_query("what is the refund policy")
print(len(vec)) # 1536 dimensions

# fully local alternative, no API cost
from langchain_huggingface import HuggingFaceEmbeddings
embeddings = HuggingFaceEmbeddings(
    model_name="BAAI/bge-small-en-v1.5")
```

### RULES THAT SAVE YOU PAIN



Use the same model for indexing and querying, always



Changing the model means reindexing everything

# Store in a Vector DB

INDEX

## WHAT THIS STEP IS

The vector database holds your embeddings and finds nearest neighbours in milliseconds. Persist it so ingestion runs once, not on every app start.

```
from langchain_chroma import Chroma

store = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    collection_name="handbook",
    persist_directory="./chroma_db",
)

# later runs just reopen it
store = Chroma(
    collection_name="handbook",
    embedding_function=embeddings,
    persist_directory="./chroma_db")
```

## CHOOSING ONE



Chroma or FAISS for local



Pinecone or Qdrant at scale



pgvector if you already run  
Postgres



Check metadata filter  
support

# The Retrieve Node

SEARCH

## WHAT THIS STEP IS

Your first real graph node. It takes the query from state, pulls candidate chunks from the store and writes them back into state.

```
retriever = store.as_retriever(  
    search_type="mmr",          # diversity aware  
    search_kwargs={"k": 12, "fetch_k": 40},  
)  
  
def retrieve(state: RAGState) -> dict:  
    q = state.get("query") or state["question"]  
    docs = retriever.invoke(q)  
    return {"docs": docs, "query": q}
```

## READING THE CODE



mmr fetches 40 then keeps 12 diverse chunks, not 12 near duplicates



Retrieve wide here, then narrow it down in the reranking step

# The Rerank Node

PRECISION

## WHAT THIS STEP IS

Embedding search is fast but rough. A cross encoder reads the question and each chunk together and scores true relevance, then you keep only the best few.

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder(
    "cross-encoder/ms-marco-MiniLM-L-6-v2")

def rerank(state: RAGState) -> dict:
    docs = state["docs"]
    pairs = [(state["query"], d.page_content) for d in docs]
    scores = reranker.predict(pairs)
    ranked = sorted(zip(scores, docs), key=lambda x: -x[0])
    return {"docs": [d for _, d in ranked[:4]]}
```

## WHY THIS SINGLE STEP LIFTS QUALITY MOST



Fewer but sharper chunks means less noise in the prompt



Smaller context costs fewer tokens and answers faster

# Build the Prompt

## GROUNDING

### WHAT THIS STEP IS

The prompt is where you force grounding. Give the model the context, tell it to use nothing else and give it permission to say it does not know.

```
from langchain_core.prompts import ChatPromptTemplate

SYSTEM = """You answer only from the context below.
If the answer is not in the context, say
'I could not find this in the documents.'
Cite the source file and page for every claim."""

prompt = ChatPromptTemplate.from_messages([
    ("system", SYSTEM),
    ("human", "Context:\n{context}\n\nQuestion: {question}"),
])

def format_ctx(docs):
    return "\n\n".join(
        f"[{d.metadata['source']}]
p[{d.metadata['page']}]}\n{d.page_content}"
        for d in docs)
```

Tagging each chunk with its source inside the context is what makes citations possible. The model can only cite what it can see.

# The Generate Node

LLM

## WHAT THIS STEP IS

Feed the reranked context and the question into the model. Keep temperature low so the model stays close to the retrieved text instead of inventing.

```
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
chain = prompt | llm | StrOutputParser()

def generate(state: RAGState) -> dict:
    answer = chain.invoke({
        "context": format_ctx(state["docs"]),
        "question": state["question"],
    })
    return {"answer": answer}
```

## READING THE CODE

→ The pipe operator chains prompt, model and parser into one runnable

💡 Temperature zero is the right default for factual document QA

# Wire the Graph

## ASSEMBLY

### WHAT THIS STEP IS

Every node is a plain function. LangGraph only needs to know the order they run in and where the flow starts and ends.

```
from langgraph.graph import StateGraph, START, END

builder = StateGraph(RAGState)
builder.add_node("retrieve", retrieve)
builder.add_node("rerank", rerank)
builder.add_node("generate", generate)

builder.add_edge(START, "retrieve")
builder.add_edge("retrieve", "rerank")
builder.add_edge("rerank", "generate")
builder.add_edge("generate", END)

graph = builder.compile()
out = graph.invoke({"question": "What is the refund window?"})
print(out["answer"])
```

You now have a working RAG app. The next three slides turn it into a system that catches its own mistakes.

# Grade the Retrieved Docs

ADVANCED

## WHAT THIS STEP IS

Before generating, ask a small model whether the retrieved chunks can actually answer the question. This is the check that makes a self correcting loop possible.

```
from pydantic import BaseModel, Field

class Grade(BaseModel):
    verdict: str = Field(description="useful or not_useful")

grader = llm.with_structured_output(Grade)

def grade_docs(state: RAGState) -> dict:
    ctx = format_ctx(state["docs"])
    r = grader.invoke(
        f"Question: {state['question']}\nContext: {ctx}\n"
        f"Can this context answer the question?"
    )
    return {"grade": r.verdict}
```

## READING THE CODE

✔ with\_structured\_output forces a clean field instead of free text

🔄 Use a cheap fast model here, it runs on every query

# Add the Rewrite Loop

SELF CORRECTING

## WHAT THIS STEP IS

If the grade says the context is weak, rewrite the query and retrieve again. This branch is the thing a plain LangChain chain cannot do.

```
def rewrite(state: RAGState) -> dict:
    better = llm.invoke(
        f"Rewrite for better document search:
        {state['question']}"
    )
    return {"query": better.content,
            "retries": state.get("retries", 0) + 1}

def route(state: RAGState) -> str:
    if state["grade"] == "useful" or state["retries"] >= 2:
        return "generate"
    return "rewrite"

builder.add_conditional_edges("grade_docs", route,
    {"generate": "generate", "rewrite": "rewrite"})
builder.add_edge("rewrite", "retrieve")
```

Retrieve

,

Grade

,

Rewrite  
loops back

,

Generate

# Add Memory and Streaming

PRODUCTION

## WHAT THIS STEP IS

A checkpointer saves graph state after every node. That gives you multi turn conversations, resume after failure and the ability to inspect any past run.

```
from langgraph.checkpoint.memory import MemorySaver

graph = builder.compile(checkpointer=MemorySaver())
cfg = {"configurable": {"thread_id": "user-42"}}

graph.invoke({"question": "What is the refund window?"}, cfg)
graph.invoke({"question": "And for digital goods?"}, cfg)

# stream node by node instead of waiting
for step in graph.stream({"question": q}, cfg):
    print(step)
```

## IN REAL DEPLOYMENTS



Swap in a Postgres saver



Add human approval nodes

# Evaluate the Pipeline

MEASURE

## WHAT THIS STEP IS

Without numbers you are guessing. Build a small question set with expected answers and score retrieval and generation separately, so you know which half broke.

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall)
from datasets import Dataset

rows = []
for q, truth in testset:
    out = graph.invoke({"question": q})
    rows.append({"question": q, "answer": out["answer"],
                 "contexts": [d.page_content for d in out["docs"]],
                 "ground_truth": truth})

print(evaluate(Dataset.from_list(rows), metrics=[
    faithfulness, answer_relevancy,
    context_precision, context_recall]))
```



Low context recall means fix retrieval



Low faithfulness means fix the prompt

# The Whole Pipeline



Load



Chunk



Embed



Store



Retrieve



Rerank



Grade



Rewrite



Generate



Evaluate

**Anyone can call a retriever.  
Engineers build the loop that  
checks its own work.**

Save this for your next RAG build. Follow for more.